

Politecnico di Milano

Anno accademico 2010-2011

Ingegneria del Software – Appello del 4 luglio 2011

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Il back-end di un sistema per la vendita on-line di biglietti per concerti, rappresentazioni teatrali ed eventi sportivi definisce una classe `Evento`:

```
public class Evento {
   ...
   public ArrayList<Posto> selezionaPosti(int num, int tipo)
   public boolean acquistaPosti(ArrayList<Posto> p)
   public int numFile()
   public Posto[] fila(int n)
   public int postiDisponibili()
}
```

- `selezionaPosti` propone i migliori `num` posti disponibili per l'evento scelto. Per semplicità, si supponga che i posti siano divisi per tipo: 0 se il posto non è numerato e 1 se lo è. Per i posti di tipo 0 non c'è alcun controllo. Per i posti di tipo 1, il sistema deve invece sempre restituire la soluzione migliore possibile. I posti sono tutti ordinati per file e `p1` è migliore di `p2` se `p1.fila()` $<$ `p2.fila()` (il metodo `fila()` è definito per la classe `Posto`, come si vedrà più avanti). Il metodo (`selezionaPosti`) non solleva eccezioni, ma è invocabile solo se la disponibilità di posti per l'evento non è esaurita.

- `acquistaPosti` consente l'effettivo acquisto dei biglietti richiesti. Il metodo deve controllare che quanto richiesto sia disponibile prima dell'esecuzione del metodo, mentre al termine dell'esecuzione i posti richiesti devono risultare occupati. Il metodo fornisce un risultato booleano che indica l'esito dell'operazione.

- `numFile` è un metodo puro che restituisce il numero di file di posti disponibili per l'evento.

- `fila` è un metodo puro che ritorna i posti della fila selezionata.

- `postiDisponibili` è un metodo puro che restituisce i posti disponibili per l'evento in quel momento.

La classe `Posto` offre il metodo `boolean disponibile()` per sapere se il posto richiesto è disponibile. Se il posto richiesto non dovesse esistere, la risposta sarebbe comunque `false`. `Posto` offre anche il metodo `int fila()` che restituisce la fila a cui si trova il posto.

Si scrivano in JML le pre- e post-condizioni dei metodi `selezionaPosti` e `acquistaPosti`.

Soluzione: una specifica non eseguibile e' la seguente:

```
//@assignable /nothing;
//@requires (tipo == 0 || tipo == 1) &&
            (0 < num <= postiDisponibili());
//@ensures \result.size() == num &&
          (\forall Posto p; \result.contains(p); p.disponibile()) &&
          (tipo == 1) =>
               !(\exists Posto p;
                   \result.contains(p);
                   (\exists int f;
                   0<f && f<p.fila();
                   (\exists Posto q;
                         fila(f).contains(q);
                         q.disponibile() && !\result.contains(q))
                   )
          );
ArrayList<Posto> selezionaPosti(int num, int tipo);

//@ requires p != null &&
//@          (\forall Posto posto; p.contains(posto); posto != null);
//@ ensures  (\result  <==>
              (\forall Posto posto;
                p.contains(posto);
                \old(posto.disponibile) && !posto.disponibile()
              )
            ) &&
            (\forall Posto posto;
              !p.contains(posto);
               posto.disponibile() <==> \old(posto.disponibile())
            )
boolean acquistaPosti(ArrayList<Posto> p);
```

Esercizio 2

Un `Orto` contiene diversi ortaggi (definiti dalla classe `Ortaggio`). Normalmente gli ortaggi sono disponibili tutto l'anno (ortaggi "generici"), ma alcuni sono stagionali (`OrtaggioStagionale`). Le due tipologie d'ortaggio definiscono un metodo `raccogli()` senza parametri per raccogliere l'ortaggio destinatario del messaggio di invocazione. Si vuole discutere se, rispettando il principio di sostituzione, `OrtaggioStagionale` debba essere definito come sotto-classe di `Ortaggio` o vice-versa. Nella discussione, si può introdurre se necessario un metodo pubblico e puro `pronto()` che restituisce vero se l'ortaggio è pronto per essere raccolto.

Si considerino, in particolare, le seguenti alternative per l'operazione `raccogli()`:

1. Se `raccogli` dovesse essere sempre definita come funzione "totale", ovvero venisse restituita un'eccezione qualora l'ortaggio non fosse pronto per il raccolto, quale gerarchia di ereditarietà potremmo immaginare? Perchè? Come si esprimerebbe in JML la specifica di `raccogli` nelle due classi?

2. Se si preferisse invece definire `raccogli` come una funzione parziale nel caso si volesse raccogliere un `OrtaggioStagionale` fuori stagione, quale gerarchia di ereditarietà potremmo immaginare? Perchè? Come si scriverebbe in JML la specifica di `raccogli` nelle due classi?

Soluzione Punto 1

La specifica di Ortaggio deve essere

```
public class Ortaggio{
        //@ ensures  (* this e' raccolto *);
        public void raccogli();
}
```

mentre quella di OrtaggioStagionale sarebbe:

```
public class OrtaggioStagionale{
        //@ ensures  pronto() && (* this e' raccolto *);
        //@ signals  (NonProntoException e) !pronto() && !(* this e' raccolto *);
        public void raccogli() throws NonProntoException;

        //@ ensures  (* pronto() e' true sse this puo' essere raccolto *);
        public boolean pronto() ;


}
```

Con queste definizioni, OrtaggioStagionale non puo' essere sottoclasse di Ortaggio, perche' sarebbe violata la regola delle segnature (che non consente l'aggiunta di eccezioni a raccogli()). E' sintatticamente possibile che Ortaggio derivi da OrtaggioStagionale, ma apparentemente si violerebbe la regola dei metodi, in quanto la ensures di Ortaggio().raccogli() e' in generale piu' debole di quella di OrtaggioStagionale.raccogli() (perche' elimina la pronto()). E' possibile pero' rendere le due classi compatibili, definendo pronto(), ereditato anche in Ortaggio, in modo da valere sempre true. Basta quindi definire Ortaggio come segue:

```
public class Ortaggio extends OrtaggioStagionale{
        //@ ensures  true;
        public void pronto();
}
```

Quindi, questa versione di Ortaggio non ridefinisce raccogli() (e quindi non puo' violare la regola dei metodi).
Si noti che la ridefinizione di pronto() in Ortaggio non viola la regola dei metodi, in quanto ogni ortaggio stagionale ha sempre la possibilita' di essere pronto().

Soluzione Punto 2

```
public class OrtaggioStagionale{
        //@ requires pronto();
        //@ ensures  raccolto();
        public void raccogli();
}
```

Se OrtaggioStagionale fosse erede di Ortaggio, allora il principio di sostituzione sarebbe violato, in quanto OrtaggioStagionale.raccogli ha una precondizione piu' forte di quella di Ortaggio.raccogli. Come nel caso precedente, tuttavia, Ortaggio puo' essere definito come erede di OrtaggioStagionale, sempre ridefinendo il solo metodo pronto() (come true):

```
public class Ortaggio extends OrtaggioStagionale{
        //@ ensures  true;
        public void pronto();
}
```

Esercizio 3

Si consideri il metodo seguente e si generi un insieme di casi di test che soddisfi il criterio di copertura delle diramazioni (*branch coverage*)

```
0   public int proc (int x , int y ) {
1        int n;
2        if ( x==y ) {
3             return 1;
4        } else {
5             if ( x>y ) {
6                  y=x*x;
7                  n=0;
8             } else {
9                  n=1;
10           }
11           while ( x<y ) {
12                n=n+1;
13                x=x+2;
14           }
15      }
16      return n;
17 }
```

Soluzione: bastano tre casi di test, in modo da attraversare le diramazioni 3, 4, 6 9, 12, 15 Ad esempio: (1,1) (percorre 0,1,2,3), (2,1) (percorre 0,1,2,4,5,6,7,11,15,16), e (1,2) (percorre 0,1,2,4,5,8,9,10,11,12,13,14,11,15,16)

Esercizio 4

Un'applicazione per ascoltare musica dalla rete consente a ogni utente (account) di caricare i propri brani e di ascoltarli da un qualsiasi dispositivo. Un brano è identificato da un nome, una durata, un artista, un album, un genere e un contenuto (il brano vero e proprio). Ogni utente ha un nome, una password e un livello di servizio (base, avanzato, professionale). Ogni utente può registrare un numero massimo di 5 dispositivi con cui ascoltare la propria musica. Inoltre, ad ogni utente è sempre associata una e una sola collezione di brani, mentre l'utente può decidere di visualizzarla e usarla in modi diversi, ordinando i diversi brani per artista, per genere, per album, oppure random. L'utente deve poter scegliere la politica dâaccesso a piacimento e deve anche poterla cambiare liberamente mentre ascolta la propria musica.

Si definisca un diagramma delle classi UML per rappresentare la specifica sopra riportata. Si consiglia di adottare opportuni design pattern, motivandone l'uso.

Traccia della soluzione: Definire le classi Account, Dispositivo, CollezioneBrani, Brano, e un'interfaccia LivelloDiServizio, implementata dalle tre classi LivelloBase, LivelloAvanzato, LivelloProfessionale. L'implementazione di queste tre classi potrebbe utilizzare un singleton. Si completa con opportune relazioni di associazione (possiede), usa, DispositiviRegistrati, contiene, ecc, e relative cardinalita'.
Per quanto riguarda l'accesso, si potrebbe utilizzare il pattern Strategy, definendo una classe astratta VistaBrani e quattro classi concrete (VistaPerArtista, VistaPerGenere, VistaPerAlbum, VistaRandom). La VistaBrani e' in una relazione "mostra" con la classe Brano, ed e' usata dalla classe Account per stabilire la VistaInUso (che puo' quindi cambiare dinamicamente).